

Website Analytics Public How-To Guide

Build, configure, and safely publish the Delphi FMX GA4 dashboard

Last changed: July 23, 2026

Public repository edition draft

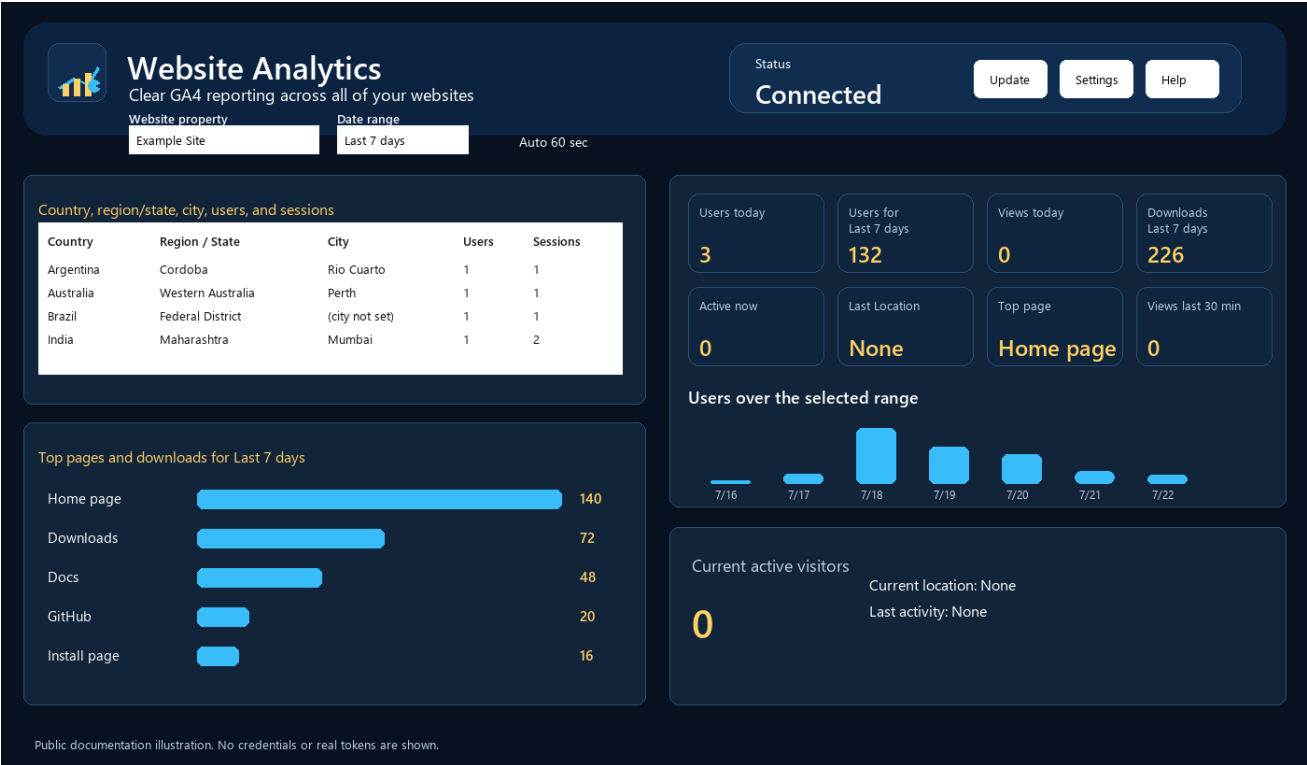
Table of Contents

- Table of Contents 2
- Purpose of this guide..... 3
- Before the repository is made public..... 3
- Clone and open the source 4
- Expected folder layout 4
- Local SQLite settings database 4
- Create Google Cloud OAuth credentials 5
- Find the GA4 property ID 5
- First run checklist 5
- Common setup errors 6
- GitHub authentication for contributors 6
- Public release checklist for maintainers..... 6

Purpose of this guide

This guide is written for a programmer who downloads the source code from a future public GitHub repository and wants to build the app, configure their own Google Analytics 4 properties, and run the dashboard without using any private developer credentials.

- The repository should be treated as source code only; every user supplies their own Google Cloud OAuth desktop client and their own GA4 property IDs.
- The app reads GA4 report data live through the Google Analytics Data API. Analytics report rows are memory-only and are discarded when the app closes.
- Local settings live in a SQLite database created at startup from the tracked schema file. The generated SQLite file must stay out of Git.



Public-safe dashboard illustration showing the app layout without credentials or private tokens.

Before the repository is made public

Do not simply flip a private development repository to public if it ever contained private databases, OAuth secrets, refresh tokens, credential files, or personal development paths. The safest public release path is a fresh repository populated from a clean export.

Keep	Remove or replace before publishing
Delphi .pas, .fmx, .dpr, .dproj files	databases/*.sqlite3 and any token cache
databases/WebsiteAnalytics.schema.sql	OAuth client secrets, refresh tokens, credential JSON, .env, PEM/key files
README, docs, help, intended images/icons	personal paths such as a developer user profile or private backup path
build scripts and .gitignore	bin, dcu, temporary files, IDE caches, accidental logs

Public repository safety model

Track in Git

- Delphi source and editable FMX forms
- README, guides, help files
- databases/WebsiteAnalytics.schema.sql
- images/icons intended for distribution
- .gitignore and clean build scripts

Never track in Git

- databases/*.sqlite3
- OAuth client secrets or refresh tokens
- credential JSON, .env, PEM, key files
- personal machine paths or user profiles
- bin/dcu/generated build output

Public repository safety checklist.

Clone and open the source

1. Install Embarcadero RAD Studio or Delphi with FireMonkey desktop support. The original development project used the RAD Studio 37.0 toolchain, but public users should use the supported version documented by the repository at release time.
2. Open a command prompt or Git Bash in a neutral source folder such as C:\Source.
3. Clone the future public repository: `git clone https://github.com/<your-user>/<repo>.git`
4. Open WebsiteAnalytics.dproj in RAD Studio. Avoid editing generated build output folders directly.
5. Build Win64 Debug first. If needed, build Win32 Debug after Win64 is working.

Expected folder layout

Folder/File	Purpose
WebsiteAnalytics.dpr / WebsiteAnalytics.dproj	Delphi application entry point and project file.
WebsiteAnalytics.MainForm.pas/.fmx	Main FMX dashboard UI and behavior. The form should remain editable in the IDE.
WebsiteAnalytics.GA4DataModule.pas	Google Analytics Data API request and response handling.
WebsiteAnalytics.AuthenticationDataModule.pas	Desktop OAuth, access token refresh, local loopback redirect, and DPAPI refresh-token protection.
WebsiteAnalytics.SettingsDataModule.pas	SQLite settings database creation, defaults, and read/write access.
databases/WebsiteAnalytics.schema.sql	Tracked schema/default settings used to create a fresh local database.
docs and help	User, engineering, how-to, and local HTML help documentation.

Local SQLite settings database

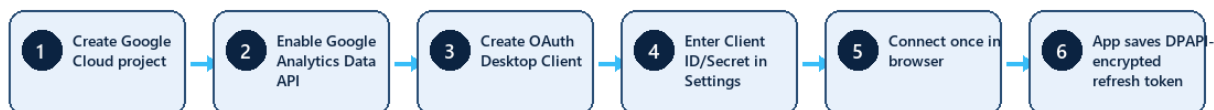
The public repository should track the schema SQL, not a real SQLite database. On first run, the app creates or updates the local settings database. That local database belongs to the user running the program.

- Tracked: databases/WebsiteAnalytics.schema.sql.

- Ignored: databases/*.sqlite3.
- Stored locally: website definitions, GA4 property IDs, dashboard defaults, OAuth client ID/secret entered by the user, saved grid widths, and the DPAPI-encrypted refresh token.
- Not stored locally as analytics history: report rows, location rows, page/action rows, graph rows, and realtime rows. Those remain memory-only.

Create Google Cloud OAuth credentials

Google OAuth and GA4 setup flow



Public repository rule: every programmer creates and enters their own Google OAuth desktop credentials.

Do not commit credentials, databases, tokens, or credential JSON files.

Each public user supplies their own Google Cloud and GA4 credentials.

6. Go to Google Cloud Console and create or choose a project owned by the user or organization running the app.
7. Enable the Google Analytics Data API for that project.
8. Configure the OAuth consent screen. For private testing, add the Google accounts that will sign in as test users. For broader distribution, plan for Google verification.
9. Create an OAuth 2.0 Client ID with application type Desktop app. The OAuth Client ID is not a Gmail address.
10. Copy the Desktop OAuth Client ID and Client Secret into the app Settings screen.
11. Use the loopback redirect URI shown in the app, typically <http://127.0.0.1:53682/oauth2redirect>.
12. Click Connect Google in Settings and approve the read-only Analytics scope.

Find the GA4 property ID

Use the numeric GA4 property ID. Do not use the Measurement ID that starts with G-, do not use the Google Cloud project number, and do not use a website URL.

13. Open Google Analytics.
14. Select the GA4 property for the website.
15. Open Admin.
16. Find Property details or the property selector area that shows the numeric property ID.
17. Copy that numeric value into the app Properties screen.

First run checklist

18. Open the app.

19. Click Settings and enter the OAuth desktop client ID, client secret, and loopback port if different from the default.
20. Click Properties and add each website with display name, website URL, numeric GA4 property ID, color, enabled state, and order.
21. Click Connect Google once. Complete the browser sign-in.
22. Return to the app and click Update.
23. Leave Auto 60 sec checked if you want the dashboard to refresh while it remains open.

Common setup errors

Symptom	Likely cause	Fix
403: API has not been used	Google Analytics Data API is not enabled in the Google Cloud project.	Enable the API and retry.
403: insufficient permissions	Wrong GA4 property ID or signed-in account lacks GA4 access.	Verify numeric property ID and GA4 Admin access.
OAuth client ID needed	Settings does not contain a desktop OAuth client ID.	Create/copy the desktop client ID from Google Cloud.
Google login appears every startup	Refresh token missing, revoked, cannot decrypt, or the user clicked Disconnect.	Reconnect from Settings.
No rows returned	The date range/property has no GA4 data or GA4 suppressed/omitted fields.	Try a wider date range and confirm GA4 is receiving events.

GitHub authentication for contributors

Public users who only download source do not need push access. Contributors or maintainers who push changes need GitHub authentication through Git Credential Manager, GitHub Desktop, or the GitHub CLI.

24. Install Git for Windows.
25. Confirm `git --version` works.
26. If using GitHub CLI, run `gh auth login` and choose GitHub.com, HTTPS, and browser login.
27. If using normal Git, the first push may open Git Credential Manager in a browser.
28. Never paste OAuth client secrets or refresh tokens into commits, issues, screenshots, or pull requests.

Public release checklist for maintainers

- Create a clean public repository from a scrubbed export, not from private history that may contain secrets.
- Run `git status` and inspect every staged file before commit.
- Search the tree for `client_secret`, `refresh_token`, `.sqlite3`, `.env`, `.json` credential names, private drive letters, and personal user folders.
- Build Win64 and Win32, then smoke-test startup with no private database.
- Confirm docs explain that every user brings their own Google Cloud OAuth desktop client.